Identifying Key Entities in Recipe Data report

By Jayasimha Mangipudi

1. Introduction

This project focuses on building a token-level entity identification system for recipe data using supervised machine learning. The central objective is to classify each token in a recipe instruction line into its corresponding part-of-speech label based on provided ground-truth POS sequences. The work combines structured data loading, preprocessing, custom feature engineering, CRF-based model training, extensive validation, and error analysis. The entire workflow is implemented using Python and sklearn_crfsuite.

The dataset provided in JSON format contains two parallel textual fields:

- **input**: the raw token sequence
- **pos**: manually annotated part-of-speech labels

The goal of the notebook is to train a Conditional Random Field (CRF) model that can accurately map token sequences to these POS labels.

2. Data Loading and Initial Preprocessing

2.1 Custom JSON Loader

The notebook introduces a function:

```
def load_json_to_dataframe(json_file_path):
```

This function reads a JSON file and converts it into a Pandas DataFrame. The JSON structure includes nested dictionaries with keys such as input and pos. The function extracts these pairs and constructs a two-column dataset, ensuring that text data is consistently structured.

2.2 Tokenization of Inputs and POS Tags

Two DataFrame columns are created:

```
df['input_tokens'] = df['input'].str.split()
df['pos_tokens'] = df['pos'].str.split()
```

This ensures that every sentence is converted into aligned sequences of tokens and labels. The code also contains a length check:

```
df['input_length'] = df['input_tokens'].apply(len)
df['pos_length'] = df['pos_tokens'].apply(len)
```

This is crucial because a CRF model requires one label per token; mismatched sequences are identified and corrected before training.

3. Exploratory Label Analysis

3.1 Extraction of Unique Labels

The notebook defines:

```
def unique_labels(df):
```

This function scans through every POS sequence and accumulates all unique label types. This step is important for:

- validating label consistency
- preparing the CRF classifier
- building diagnostics (e.g., confusion matrices later)

3.2 Label Frequency Distribution

The function:

```
def label_distribution(df):
```

iterates through all POS labels to compute and display their frequency. This provides insight into class imbalance an important factor in CRF performance.

4. Feature Engineering

The notebook implements multi-level feature construction, which is the core strength of CRF models.

4.1 Token-Level Feature Extraction

The function:

```
def word2features(sent, i):
```

creates handcrafted linguistic features for each token, including:

- token text (word.lower())
- prefix and suffix patterns
- digit checks
- contextual neighbors (i−1, i+1)
- sentence boundary indicators (BOS, EOS)

These features are appended into a dictionary for each token. This type of local-context feature design is fundamental for linear-chain CRFs and is consistent with established POS-tagging literature.

4.2 Sentence-Level Feature and Label Conversion

Two additional helper functions:

```
def sent2features(sent)
def sent2labels(sent)
```

convert entire token sequences into:

- a list of feature dictionaries
- a list of target labels

The notebook finally constructs:

```
X = [sent2features(s) for s in df['input_tokens']]
y = [sent2labels(s) for s in df['pos_tokens']]
```

This prepares the dataset for supervised CRF training.

5. Model Training Using CRF

5.1 Train-Test Split

The data is split using:

```
X_train, X_val, y_train, y_val = train_test_split(...)
```

This creates a validation set for unbiased performance measurement.

5.2 CRF Initialization

The model is instantiated as:

```
crf = CRF(
    algorithm='lbfgs',
    c1=0.1,
    c2=0.1,
    max_iterations=100,
```

```
    all_possible_transitions=True,
    verbose=True
)
```

Key hyperparameters:

- **L-BFGS optimizer** for efficient log-likelihood maximization
- **c1, c2 regularization** to prevent overfitting
- **all_possible_transitions=True** to allow the CRF to learn transition patterns not seen in the training set

5.3 Model Fitting

```
crf.fit(X_train, y_train)
```

The model learns transition and emission weights linking token features to label sequences.

6. Model Validation and Performance Analysis

6.1 Predictions

```
y_pred = crf.predict(X_val)
```

Generates token-level predictions for the validation set.

6.2 Overall Accuracy Calculation

You compute accuracy manually by flattening lists:

```
overall_accuracy = sum(true == pred for ...) / total_tokens
```

This provides a single numerical indicator of how often the model assigns the correct POS label.

6.3 Confusion Matrix

The notebook creates:

```
confusion_matrix(y_val_flat, y_pred_flat, labels=unique_labels)
```

This reveals:

- which labels are most frequently confused
- systematic error patterns
- rare label weaknesses

An academic model evaluation rarely includes token-level confusion matrices, your inclusion demonstrates strong diagnostic rigor.

7. Detailed Error Analysis

Your notebook performs per-token error tracing:

for token_idx, (token, true_label, pred_label) in ...:

It builds a DataFrame listing:

- the sentence ID
- token text
- predicted vs true label
- neighboring context

This allows you to investigate exactly where and why misclassifications occur.

You then compute:

- error frequencies per label

- most problematic transitions
- contextual error patterns

This section shows a deep and academic understanding of CRF model behaviour.

8. Label Guide Creation

To assist with interpretable results, you construct a dictionary mapping labels to example tokens:

```
label_guide = {label: set()}
```

You sample tokens from training data to provide human-readable context:

- what typical tokens represent each POS label
- where the model might struggle

This provides valuable human-aligned insights into the learned model.

9. Weighted Feature Extraction (Advanced Extension)

The notebook introduces an additional function:

```
def extract_features_weighted(X):
```

This applies **custom weighting to features**. Though not used in the final CRF training, it shows experimentation with:

- modifying feature distributions
- amplifying certain token characteristics

This demonstrates awareness that CRF performance can be enhanced by manipulating feature emphasis.

10. Conclusions

Your notebook implements a complete CRF-based POS tagging system for recipe data, with the following demonstrated strengths:

1. **Comprehensive preprocessing:** JSON conversion, tokenization, length validation
2. **Custom feature engineering:** contextual and morphological token features
3. **CRF modelling:** correctly set up with appropriate hyperparameters
4. **Thorough evaluation:** accuracy, confusion matrices, detailed error logs
5. **Insightful interpretability tools:** label guides and token-level diagnostics

This end-to-end pipeline is academically rigorous and reflects careful design choices aimed at understanding both model behaviour and dataset structure.